

Atelier 04 : Création d'un pipeline DevOps avec Jenkins

Objectifs	Outils utilisés
1. Comprendre les concepts de base d'un pipeline CI/CD. 2. Configurer Jenkins pour automatiser les étapes de build, test et déploiement. 3. Utiliser Git pour la gestion de version. 4. Conteneuriser une application avec Docker. 5. Déployer l'application sur un serveur distant.	Git : Gestion de version. Jenkins : Automatisation du pipeline CI/CD. Docker : Conteneurisation de l'application. Docker Hub : Registre d'images Docker.

Environnement requis :

- Une machine Linux (ou une VM) avec accès à Internet.
- Jenkins installé et configuré.
- Docker installé.
- Un compte Docker Hub.
- Un serveur distant (ou une VM) pour le déploiement.

Étape 1 : Préparation du projet

1. Créer un dépôt Git :

- Créez un dépôt sur GitHub ou GitLab.
- Clonez le dépôt localement :

```
git clone https://github.com/votre-utilisateur/votre-depot.git
cd votre-depot
```

2. Ajouter une application simple :

- Créez une application web simple en Python (Flask) ou Node.js (Express).
- Exemple en Python (app.py) :

```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def hello():
    return "Hello, DevOps World!"
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

- Ajoutez un fichier requirements.txt pour les dépendances : Flask==2.0.1

3. Versionner le projet :

- Ajoutez les fichiers au dépôt Git :

```
git add .
git commit -m "Initial commit"
git push origin main
```

Étape 2 : Configuration de Jenkins

1. Installer Jenkins :

- Suivez la documentation officielle pour installer Jenkins sur votre machine.

2. Créer un pipeline CI/CD :

- Dans Jenkins, créez un nouveau projet de type "Pipeline".
- Configurez le pipeline pour utiliser le dépôt Git créé précédemment.
- Ajoutez un fichier Jenkinsfile à la racine du projet :

```
pipeline {
  agent any
  environment {
    DOCKER_HUB_CREDENTIALS = credentials('docker-hub-credentials')
  }
  stages {
    stage('Build') {
      steps {
        sh 'echo "Building the application..."'
        sh 'docker build -t my-python-app .'
      }
    }
    stage('Test') {
      steps {
        sh 'echo "Running tests..."'
        // Ajoutez ici des commandes pour exécuter des tests unitaires
      }
    }
    stage('Push to Docker Hub') {
      steps {
        sh 'echo "Pushing the Docker image to Docker Hub..."'
        sh 'docker login -u ${DOCKER_HUB_CREDENTIALS_USR} -p ${DOCKER_HUB_CREDENTIALS_PSW}'
        sh 'docker tag my-python-app ${DOCKER_HUB_CREDENTIALS_USR}/my-python-app:latest'
        sh 'docker push ${DOCKER_HUB_CREDENTIALS_USR}/my-python-app:latest'
      }
    }
    stage('Deploy') {
      steps {
        sh 'echo "Deploying the application..."'
        sh 'ssh user@remote-server "docker pull ${DOCKER_HUB_CREDENTIALS_USR}/my-python-app:latest"'
        sh 'ssh user@remote-server "docker stop my-python-app || true"'
      }
    }
  }
}
```

```
sh 'ssh user@remote-server "docker rm my-python-app || true"'
sh 'ssh user@remote-server "docker run -d -p 5000:5000 --name my-python-app
${DOCKER_HUB_CREDENTIALS_USR}/my-python-app:latest"'
}
} }}
```

3. Configurer les credentials Docker Hub :

- Dans Jenkins, allez dans "Manage Jenkins" > "Manage Credentials".
- Ajoutez une nouvelle credential de type "Username with password" pour Docker Hub.

4. Lancer le pipeline :

- Poussez les changements sur Git et vérifiez que Jenkins exécute le pipeline.

Étape 3 : Conteneurisation avec Docker

1. Créer un Dockerfile :

- Ajoutez un fichier Dockerfile à la racine du projet :

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

2. Tester localement :

- Construisez l'image Docker : **docker build -t my-python-app .**
- Lancez le conteneur : **docker run -d -p 5000:5000 my-python-app**
- Vérifiez que l'application fonctionne en accédant à <http://localhost:5000>.